

The Story of Linear as told by its CTO

Tuomas Artman went from Big Tech to startup co-founder. He shares how they built of Linear, the working culture and learnings from Big Tech used at the startup.



GERGELY OROSZ

NOV 22, 2022 • PAID



90



7



Share



👋 Hi, this is Gergely with a 🔒 subscriber-only issue 🔒 of the Pragmatic Engineer Newsletter. In every issue, I cover challenges at Big Tech and startups through the lens of engineering managers and senior engineers.

[Tuomas Artman](#) is the co-founder and CTO of product tooling startup, [Linear](#). He's been a software engineer for more than 20 years, alternating between being a developer, a founder and an engineering manager. He previously worked at a company aspiring to be the "Disney of China," at Groupon, and at Uber.

I first met Tuomas when we both worked at Uber; he was a senior staff engineer on the mobile platform team and his team was creating Uber's new mobile architecture, [RIBs](#). Tuomas left Uber in 2019 after five years, to co-found Linear.

Over the past year or two, I've increasingly heard startup founders say they use Linear to manage their workflow, while software engineers I know have praised the tool's capabilities. In January, Linear's CEO Karri Saarinen [shared](#) that more than 1,000 startups pay for Linear, that the company runs on a small team of 17, and that they achieved their first profitable month two years after being founded.

Earlier this year, I reached out to Tuomas to learn more about his background and how he ended up in Silicon Valley and at Uber, why he cofounded Linear, how it operates, and any lessons he's learned.

In this issue, we cover:

1. From Finland to Silicon Valley
2. Starting a new company
3. The first hires
4. Technology stack
5. The working culture
6. Hiring
7. Big Tech learnings applied to a startup

With that, it's over to Tuomas.

1. From Finland to Silicon Valley

When I was 18, I won a place at university to study computer science. However, after a few weeks, I dropped out because I got a job at a company called To the Point, a business which produced CD-ROM presentations.

After a few years, a group of us friends founded a development agency called Valve Group. We had eight cofounders and I became the CTO. We built rich media presentations, and I picked up technologies like Flash and Flex on the go. In a short space of time, we became the de facto rich media house in Finland. I did this for nine years, from 2000 until 2009, by when I had grown bored and wanted to try something new.

I worked at a few startups in Finland and also founded a small consultancy. Then in 2011, I took a leap and moved to China to be CTO of a company called Apple Toon, which wanted to become the "Disney of China." When I joined, the company employed about 250 people.

My adventure in China was interesting; it was an unusual setup in many ways. For example, none of the company's computers were connected to the internet, because they feared theft. This was because an earlier team of theirs had worked on a game for a year and had it almost finished. But as launch day approached, the team building the game stole the software and launched it themselves.

Later and out of the blue, I got the opportunity to move to the US. A former colleague lived in San Francisco and needed more engineers, so he called me up. It turned out Groupon was buying his company, and my acquaintance was incentivised to bring in more engineers as part of the deal. The deal went through and Groupon acquired my Finnish company which I was using to publish iOS applications, which is how I ended up there.

Groupon

After this acquisition, I got transferred to the US on a manager visa in 2012.

I joined Groupon right after its 2011 IPO when momentum was still strong and before it started going downhill. Back then, Groupon wanted to create a new product to take on Square. To put a team in place that would build payment functionality, Groupon initially planned to buy five smaller startups, but eventually it only bought the one I'd joined via my former colleague. So, instead of having around 50 engineers to build a competitor to Square, there were only four of us. My team built a few prototypes, but in the end we merged with the Point-of-Sale team.

After two years, I got my green card, meaning I was free to work wherever I wanted. My number one choice was Uber.

So, in 2014 I joined Uber in San Francisco.

Uber

I started developing on iOS back in 2008, when the iPhone came out. I built an RSS reader early on, and it made quite a bit of money because it was one of the first apps of its type in the App Store. From then, I was hooked and created a bunch of apps on the side. I built iOS apps, as my main jobs were more backend-related. In China, I built a sync engine, and at Groupon I started out on the backend, but after merging with the Point-of-Sale team I worked on iOS full time. When I interviewed at Uber and I wanted to work on customer facing mobile, but quickly after joining the mobile platform team was formed and I was asked to manage the team. Later I moved back to being an individual contributor (IC) working as a senior staff engineer.

2. The motivation to start a company

Gergely here. Tuomas had a great career at Uber, growing into one of the most respected mobile software engineers at the company. I asked him why he left a good, cushy job.

The motivation to start a company

I live for startups and love working at small companies. I'd founded a company in Finland and knew I wanted to do it again.

I consciously took jobs in Silicon Valley to prepare for my next startup. I wanted to learn, to build up pedigree and to create a financial buffer. Most important was the learning; I wanted to gain knowledge. For me, working in Silicon Valley was the best schooling I could get as a software engineer. It was not until later that I appreciated the significance of the pedigree which comes from working at high-profile tech companies.

Jori and Karri - the two other co-founders of Linear - had also created a company called Kippt, which got funding from Y Combinator. It was a bookmarking service which grew into a knowledge base solution for corporations and eventually was [acquired by](#) Coinbase.

So, how did I meet my co-founders? I'd met Jori a long time ago, back when I was working at a startup in Helsinki. He was interested in startups and through friends, he pinged me, asking if I wanted to talk about startups. I was like, "yeah, I'm having a beer. Come over." He joined us and we were talking from then onwards.

Karri and I met in San Francisco. When you move abroad to a new city as a Finn, you tend to quickly get to know your compatriots also living there. We're both Finnish so we hung out. We were both doing well in our respective jobs, but wanted to launch another startup and at some point, we started talking about doing something together.

As a founder, if you want getting investment to be relatively easy, work at a big company first. After that, you're on the radar of investors. And if you have a founding team with Big Tech pedigree, the idea you're pitching becomes less important as a selling point. In our case, our founding team came from Airbnb, Coinbase and Uber.

Investors wanted to hear our idea, but I suspect we got funding easily because investors thought: "okay, here's the team which built products from scratch and knows how companies scale. So yeah, let's invest in them."

In the end, all three of us – Jori, Karri, and myself – are entrepreneurs at heart. Having a "cushy job" was like taking a breather between founding startups.

The idea of Linear

In early 2018, we started talking about how bad all the issue tracking solutions were and arrived at two separate realizations about project management solutions.

1. ICs dislike their current project management solutions. All big project management solutions were built for management. As a manager, these solutions work pretty decently and cater for their use cases. This was why we didn't hear many managers complain.

However, many software engineers and ICs disliked all of them; few talked positively about any of the solutions. We kept getting back to this and asking ourselves, "why hasn't anybody built a project management tool that ICs like? What are we missing?"

There was clearly a business opportunity, and Atlassian had grown huge off the back of software project management. So, it seemed like a good opportunity to start working on something better.

Still, after detecting this opportunity, we weren't really sold on the idea. While we knew we could build a better project management solution for ICs, we were not excited by the thought of building *yet another* project management software.

2. Product companies don't really have any methodology for building products.

We worked at companies regarded as among the best product companies in Silicon Valley, such as Uber, Airbnb and Coinbase. But none of them had any methodology for building products. Approaches differed from team to team. Most teams used bits and pieces from Scrum and couldn't really explain why they were working in sprints.

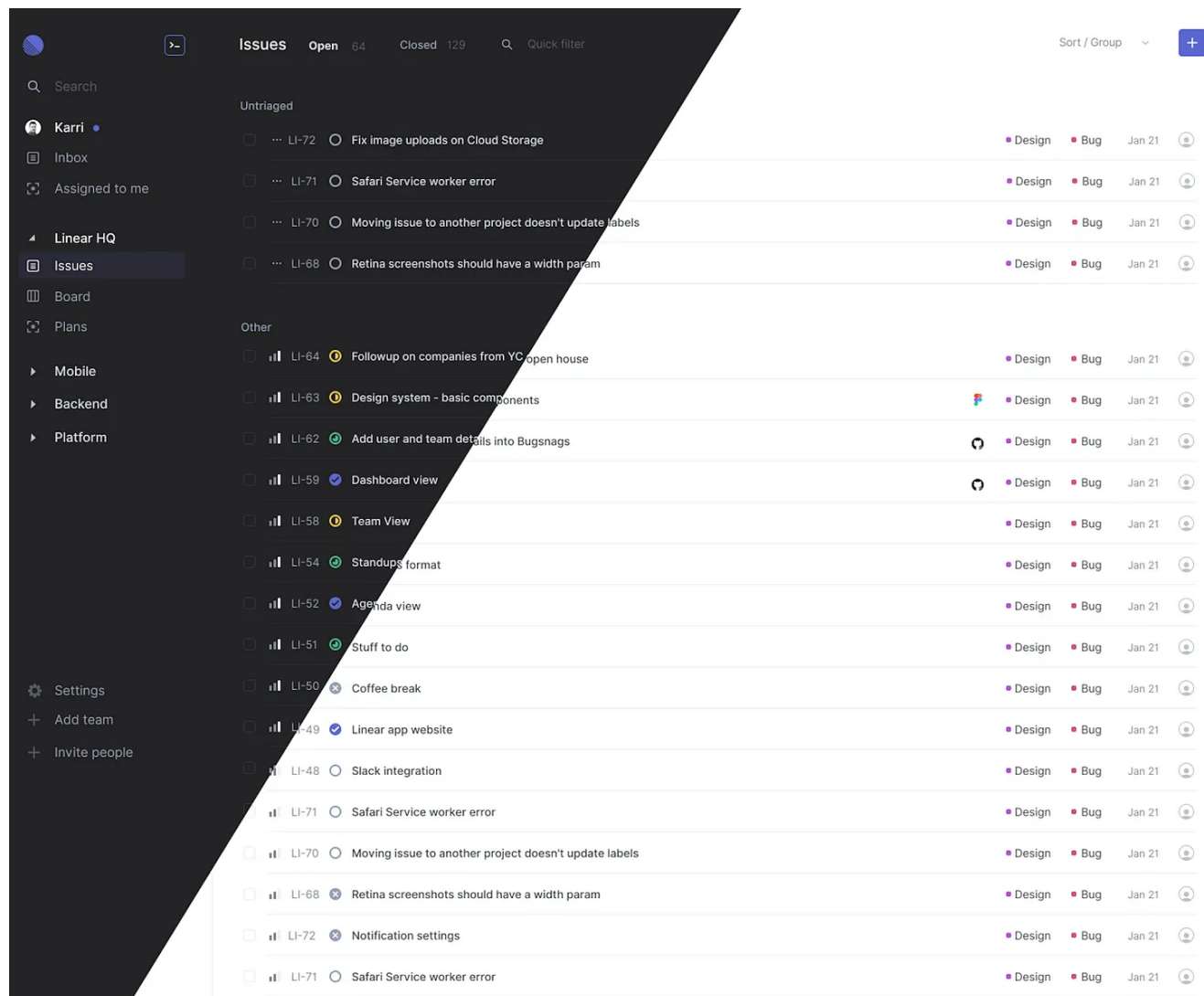
These two realizations helped formulate a mission we could finally get excited about. We weren't going to build another project management solution.

Instead, we'd do this:

Help companies build better software products. Start small with issue tracking and focus on ICs working at smaller companies, initially. As these companies grow, we'd build up more functionality and support scaleups. In the end, we'd have a product which could go head-to-head against the common enterprise solutions.

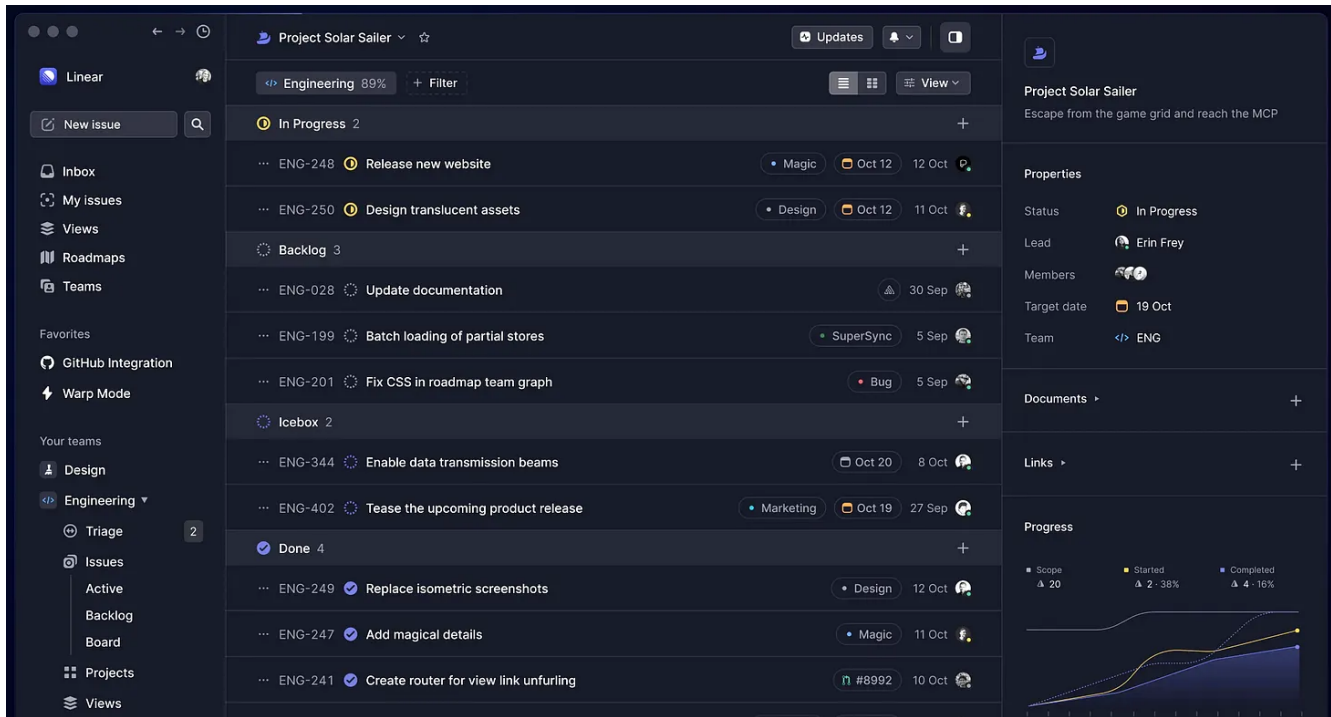
We founded the company in 2019 with two engineers and one designer. We wanted to build a simple and functional product, and all three of us shared this aspiration. Also, we complimented each other; myself and Jori are software engineers, and Karri is a designer. Jori cares deeply about the customer experience, I'm into more complex technical solutions to make the application really performant, and Karri is an excellent product designer. With this team, we thought we could create the best project management application in existence, as a start.

As a throwback, here's an early design of Linear, back in 2019:



An early design of Linear, from 2019.

As comparison, here's how Linear looks like in 2022:



Linear in 2022.

Funding

Our initial approach was not to raise funding before we had a hint of product market fit. . However, we were approached by Stephanie from Sequoia in April 2019, literally two days after we announced our company on Twitter. Because it was Sequoia, we took the meeting. To be precise, Karri did because the rest of us didn't want to get distracted from building the product.

While it would make for a great story to say Karri didn't even have a pitch deck, in reality, as I found out much later, he had put together a simple one the night before. At the meeting, Karri was clear we didn't really want to raise money until later.

Encouraged by the meeting with Sequoia, we decided to look around a bit and Karri took some meetings with other investors, which led to us receiving a few term sheets. We inspected them and decided it made no sense for us to postpone raising money because the valuation was already high for a seed round . A few months later signed with Sequoia and Index Ventures.

In the end, raising money for us was simple because we didn't want it.

3. The first hires

We raised our funding in the summer of 2019 and it took four months to make our first hire. We hired Miha, a software engineer, whom Jori and Karri worked with at Coinbase and were friends with. We had been talking with him for a while at that point.

For your first few hires, you can probably rely on your network. Reach out to people you worked with previously, whom you know will be good and compliment your team and lay the foundation for your culture. For that reason, we did no formal interviewing for the early hires.

Our next hire was Andreas, a software engineer, who started out as a contractor. Jori had followed him on Twitter for a long time and reckoned he'd be a great fit. He was finishing up a job at Klarna when we reached out. At first, we asked Andreas to help us out on the frontend. He shipped some really amazing things and we managed to convert him to full-time. This hiring approach was one way we get great people in.

Consider starting off with freelance contracts, and then hiring people. This is especially relevant in places like Europe where many of the most experienced software engineers often work as contractors, thanks to a variety of factors like higher earning potential, more favorable taxes, interesting projects and not needing to worry about the career ladder.

This was how we hired all of our recruiting team. We hired three recruiters a few months ago, who had been working with us on contracts for about two years. It was an easy decision to hire them full-time when they showed interest in joining us.

Our first five hires were three software engineers, one designer and a customer success hire. We hired Erin to help us with customer success, because we struggled with the pain of answering all the questions customers sent our way. Even when we were in beta, we'd get large amounts of email and messages through support channels. Customer support got to the point where it overwhelmed us. So, we hired Erin to start the function and to grow it, over time.

We try to hire people who can later grow into leading their respective functions. Erin is a good example of someone we hired, hoping she could grow a function as the

company developed. This means we look for early hires with the desire and capability to grow. It doesn't always work out as hoped for, but we do try our best.

This summer, we hired our first engineering manager. We approached this in pretty much the same way as with our infrastructure, in that we started to build out this function earlier than necessary. We could have probably grown quite a bit more before running into problems, but then we would have to act fast to fix them.

So, we started hiring for an engineering manager early on, in order to fill this role before we had a desperate need for it. Hiring sooner meant we could try out a few approaches with less risk, and it also gives the new hire more space to try things in post.

We currently have 30 employees, of whom 18 work on product; comprising 15 software engineers and 3 designers. The other 12 people work on support, administration, sales, marketing and recruitment.

4. Technology stack

We are building on top of Google Cloud. When you are choosing your cloud provider, you really have two straightforward options: AWS or GCP. On paper, they look pretty similar. However, in my view GCP is moving ahead on cost-for-performance.

In the end, we went with GCP because the user interface is much better than AWS.

For our tech stack, we chose a 'very normal' tech stack, based on **TypeScript**. We have **React** on the frontend and **Node** on the backend. We chose TypeScript because compiled and typed languages are more productive, and we can now also share code between the frontend and backend. We actually share code between models, and some utility code.

We use **Postgres** as a database, and **Redis** for queuing, **MongoDB** is in place for caching scenarios and **GraphQL** as our API layer. And all of it runs on **GCP with Kubernetes**.

Our proprietary [sync engine](#) plays a large role and manages data replication across clients, networking, error handling, offline mode and transactions. This makes it

possible to create even complex new features without backend engineering.

As for mobile, we don't have a mobile client out just yet, but we're working on it. We're also planning to use a 'very normal' tech stack there.

We chose to go with a 'very normal' tech stack because we wanted to ensure we don't cut any potential new hire out, based on what we ask for. This is a stack any engineer should be able to pick up quickly. And we wanted to choose a tech stack where any engineer can solo build almost any feature from start to finish.

5. The working culture

We've been full-remote since the very beginning. Because of this, we've seen the upsides and downsides of this work pattern.

Focus is one of the main benefits of remote work. There are no unnecessary meetings and pretty much everyone on the team focuses on their own work most of the time. We try to get together in-person about twice a year to do the things which are hard to do remotely, like making sure we're aligned on our long term vision.

A downside of remote is that it's harder to hire less experienced engineers. Because of remote, we only hired senior folks early on. We didn't really have a good grasp of how mentoring works in a remote environment and didn't want to do a disservice to colleagues just starting their careers.

We come together about every 6 months. Ideally, this is biannual, but we always do it at least once, in order to align on the future. We talk about things we want to achieve and do deep dives on the different teams at the company.

We work based on time zones. We have teams in the US and in Europe and organize work so that people mostly collaborate with others in the same time zone; US folks with those also in the same country, and likewise for Europe. We try to avoid having people collaborate across time zones, every day.

We use our own methodology to define and ship work. We call it [the Linear Method](#) and publicly [share](#) our philosophy, principles and practices. We believe the Linear

Method can help startups get to product-market fit, faster. A few things we do:

- Set directions with a roadmap. We do upfront planning, break it down into milestones, work on substantial projects, review the roadmap regularly and stay flexible.
- Write tasks, not user stories. We write short and simple issues which describe a task in plain language. We think user stories are an anti-pattern in product development.
- Build with users. Much of the early startup process is about learning what your customers want. We seek out users and potential users for feedback, iterate, and are flexible in meeting the demands of customers and the market.
- Write and publish a changelog. This is also a way to build in public. While it might seem silly to summarize your work in a changelog when you don't have many users, we think it's helpful and so we [publish ours](#).

A few things we do, not captured in the Linear Method:

- We don't always follow our own method as closely as we preach. We break from it when it doesn't feel like the right fit.
- Engineers are empowered to figure out what they should be working on, as the next thing.
- We tend to focus on a few major projects which everyone works on. Beyond that, people decide which smaller things to pick up, as they see fit.
- Our focus is ensuring everything we do is for the benefit of the products.

We're paying attention to how many 'misses' we have. By 'misses,' I mean things we build that might not be beneficial to the majority of our users. Our current approach still feels a bit ad hoc at times, but everything we ship does benefit the product and our customers, so we're happy. Our backlog has years' worth of work we want to build and taking any one of those ideas and shipping it will help the product.

We'll consider changing our working approach once we start having more misses. Then it will be time to think more about how we prioritize correctly. However, this has not happened yet.

6. Hiring

At Uber, I learned enough about hypergrowth in headcount that I never want to experience such a level of headcount growth again. In 2014, Uber had about 15 mobile engineers in total, split roughly evenly between iOS and Android native engineers. This number exploded to more than 400 by the time I left, about five years later. While I was there, Uber was in a constant state of hypergrowth.

There's only one path through hypergrowth and it's always very messy. Here's how it usually goes:

1. **Pre product-market fit (PMF).** You're trying to get to product-market fit and try out a bunch of different things.
2. **Finding PMF.** Suddenly, you've found product-market fit! "This is it! This will work," you say. However, at this point there's a lot of competition out there, and you need to ensure you'll win the market.
3. **Scale, fast.** You need to beat the competition, so you start to scale your business on your existing, poorly thought-out infrastructure. Remember, you built this first version of the product when you weren't sure it would even work, and were ready to throw it away at any point. Now, you need to scale it up to serve many more customers and use cases than you ever considered when building it.
4. **Hiring lots of people.** You need people to scale, fast. So, you interview candidates. You don't have a particularly high hiring bar because you need all hands on deck, now.
5. **Firefighting.** Although the new hires start to scaffold more resilient and more scalable infrastructure, you don't have time to work on it as much as you'd like. This is because you're trapped between firefighting, building new things to keep ahead of the competition, meaning you build infrastructure during the time you have left over.
6. **Room to breathe.** One or two years later, growth either stagnates or becomes more manageable. You finally have space to breathe and look around, to figure out how your systems are really doing. You always come to the same realization...

7. **Start from scratch.** As you evaluate your systems, you realize you built your products on infrastructure which is so brittle that it's a wonder it works. You realize you need to rebuild it all, starting from scratch. Now, you finally know the requirements and the constraints you need to build for.
8. **Years of rebuilding and migrating.** The rebuilding of the new system and migrating over from the old one takes much longer than you expect. At Uber, we had migrations that took years! *Read more about [migrations done well](#) in this article.*

The big problem with hypergrowth in headcount is that after a while, barely any of your engineers work on the product. I noticed that a few years into hypergrowth, it felt like 70-75% of engineers worked on infrastructure, and no one was really working on the product. *Note from Gergely; in the [interview with Ganesh Srinivasan](#), former VP of Engineering at Uber, he shared that about 20% of hires went towards the platform and he reflected that a higher allocation might have been better.*

Having a large number of people work on infrastructure is one of the major things we at Linear want to make sure that we do not do. We want to keep software engineering as a product problem. Every single engineer should be focused on making things better for the customer, not having to worry about the infrastructure.

So, we start by looking at the infrastructure and figuring out what the challenges will be, a year in advance. Then we tackle them early and are diligent about doing so. We try to put good solutions in place, instead of just fixing issues as they present themselves.

We do hiring at Linear differently from most companies. At many places, once you get an investment, you immediately start hiring. However, we wanted to do the opposite. We wanted to grow a team who we really enjoy working with, instead of growing our headcount for the sake of it.

Our aim was quality over quantity. We wanted to grow a team who are excellent at building and shipping products. We hire very few, but very good people; this is our current hiring strategy. I'd like to keep this as our strategy forever, as we really enjoy building a great team.

We've only hired senior people, so far. This is not just because we only wanted to hire seniors, it's also because of remote working. As a remote company, we didn't know how to manage more junior folks. We wanted to avoid taking risks, so we went with more experienced hires.

Our goal was to build the product, so we wanted to onboard people who don't need supervision or mentoring. Again, from the get-go, our goal was to build a team which is *really* fast at shipping.

Investing in infrastructure helped us to hire slower. We built things early on that no sane engineering team would build before hitting the product-market-fit. We did so in order to allow us to do more with fewer people, as we grow.

A good example of this is our sync engine. In the very early days of Linear, we built a complex engine which took care of synchronizing the frontend and the backend, with offline support, and all that jazz. Because we did this so early on, it meant that as we grew, engineers could focus on building features and do so at a very rapid pace as they would not have to think about networking, error management, race conditions or data synchronization between clients.

Another example was how we decided to host our backend. We decided to use GCP and use Kubernetes on top. Putting Kubernetes in place when you only have two engineers and one designer on the project sounds like bad prioritization. However, my view was that sooner or later, we'd need to move over to Kubernetes, so we might as well bite the bullet now.

In terms of why we chose Kubernetes, it felt new enough, but not too novel. We knew that if we grew, we'd eventually need to scale; we'd need to automate scaling up and scaling down, add in permissions and support other workflows. Kubernetes does this job. It was more work to do upfront, but it means we won't have to migrate.

We're incredibly picky in our hiring. Our credo has been to hire less, but hire the best. We want to build a company which you'd want to stick around at, just because of the people and taking pride in the product we build. That means keeping the team relatively

small in order to keep the work truly meaningful, instead of everyone being a small cog in a big machine.

Hiring is a key part of preserving our engineering culture. I believe that if we hire the team that we want to work with, if we hire great people, and if we limit our hiring speed, then we can onboard people who embody our engineering culture.

Linear is a much easier product to build, because we use our own product. Compare this to Uber, where we were users but we were not the primary users; the cab drivers and delivery riders.

At Linear, we're not inventing something new; we're building something better than what exists today. So, we want to work with people who care about the craft of software and the product. This is a differentiator from many other Silicon Valley companies which are inventing new things. We care more about the quality of the product, which is the most important differentiator we offer. Passion for the craft of building software is the most important thing we look for in new hires.

Our recruitment process involves very few interviews, but includes a week of working together. We have two table-stakes interviews:

1. Coding. We don't do algorithms, we do an actual problem; similar to what you would solve if you worked here.
2. Architecture. We cover how we build things and apply the same, practical approach as to coding.

Following the two interviews, we do a week-long work trial, which of course is paid. It gives us a good sense of how people enjoy aspects of our work. We give people a greenfield task to work on, which might also be shipped to production by the end of the week.

During this trial week, the candidate is part of the team. They often drive the project, getting help from other engineers and designers. We give access to the whole codebase and they take part in all meetings. The candidate gets a sense of what it's like to work with us, and vice versa.

We have been hiring slower than we could. Every now and then, us co-founders come back to the same conversation: are we too harsh in how we hire? Are we slowing down the company's growth by being too strict about how we hire?

We became profitable because we hired "too slowly." Our intention was not to become profitable early on – thanks to the funds at our disposal – but we now jokingly say that being so picky helped us get to profitability quickly.

In reality, with a bigger team, we could work on more things. Still, we always come back to our belief that it's more important to hire the right people, versus having a larger team.

Hiring and prioritizing what to work on have been the biggest challenges at Linear. Over the first four years of the company, these two challenges have consistently gone hand-in-hand. They are also somewhat self-induced. We've chosen to keep a very high bar for hiring, which has made it harder to grow our team. And because we have a small team, we've had to really prioritize our work. We have so many things we want to do and it's really painful to have to pick just a few to implement at a time.

7. Big Tech learnings applied to a startup

Having worked for many years within Big Tech, there's a few important learnings I've taken.

The larger the codebase, the more expensive it becomes to make architectural changes. I've seen this play out first-hand. At larger companies, the scale of the codebase increases to a size at which it's not only expensive, but sometimes impossible, to do large refactors or wide-ranging changes.

We know that eventually our codebase will become large, too. Knowing that the difficulty of making architectural changes will only increase, we prepare by continuously improving our architecture, code patterns and compartmentalization. We want to experience less pain when our codebase gets large and thanks to this Big Tech experience, we know what to expect.

We had to unlearn reliance on A/B tests. At Big Tech, there's a tendency to not work on anything that can't be measured. This is especially true with consumer products; after all, if you can't measure it, how do you know if you made an impact? And if you don't know whether you made an impact, why do it in the first place?

At Linear, we are building a tool, not a funnel which drives an intended outcome. This means we get little value from looking at analytics, so we use intuition and customer research.

Working at a startup involves several "wow" moments. Even though all of us ran new companies before, at Linear we've still had a few surprises after years of working at large companies.

I was surprised just *how* exciting building the first minimum viable product (MPV) is. It was this feeling of being at the very start of something extraordinary.

The rapid pace of shipping was something I'd forgotten. This is especially true after years on mobile apps, where our feedback loop from writing code to the customer using it, was around a month. I'd write code for the feature, land it, wait for it to go through internal testing and then to ship to the app store, wait for consumers to update the app, then do a staged rollout. It would be a month before I'd get feedback and analytics about how users interacted with the feature.

In contrast, during the early days of Linear it was common for us to come up with a feature, implement it, ship it and get feedback from users – all in the same day.

If you are currently working at Big Tech but hope to found a startup one day, here's some advice. Firstly: congratulations! You're already in a much better position than many others to embark on your startup journey.

If you've worked in Big Tech, anything you start will create a "ping" on the radar of venture capitalists (VCs.) If your idea is solid and you've managed to get high-caliber co-founders, raising a seed round should be pretty straight forward. For VC's, a seed round is really a bet on a team to see if it can build a "MVP" with a hint of product-market fit. For you as a founder, a seed round de-risks your journey.

One more piece of advice. Use your position at a Big Tech company to gather a following on Twitter or other social media platforms. Pick any topic which others outside of Big Tech might not have insight into, and start posting. Once you do, those Twitter followers will be your crucial first audience to kick-start whatever you're building.

Takeaways

Gergely here. Thank you Tuomas, for sharing your story and learnings. You can check out Linear [here](#). Put simply, it's an issue tracking and project management tool that teams actually enjoy using. Of course, as we've heard, this is only the beginning of a bigger vision which sees Linear not just as a better tool, but as a better **way** to build software products.

I really enjoyed catching up with Tuomas; it's refreshing to hear the story of a startup which is intentionally *not* growing fast in terms of headcount, partly because of what the founders learned from working in Big Tech. My main takeaways from our discussion are these:

Working in Big Tech can make it easier to found a startup. Pre-Linear, this is what all three founders did, and it gave them a few advantages. They were able to save up enough money to bootstrap without needing early funding. Fundraising was easier thanks to the pedigree deriving from Big Tech, and – ironically – because they did not need money.

Building large products can help you to build better small products. Tuomas values engineers working on product features and not infrastructure. To enable this, he put a platform in place which means software engineers at Linear can do exactly that. The sync engine and the auto-scaling backend on top of Kubernetes are examples.

Choosing a 'normal' tech stack is a sensible choice for startups. When Tuomas and I worked at Uber, the company used Node.JS, Python, Java and Go on the backend, while Tuomas has worked with a variety of other technologies. When starting Linear, the founding team reached for technologies which were the most 'standard' across the stack, not ones they'd used previously and were familiar with.

TypeScript on the frontend and backend allows a software engineer to build a feature end-to-end. The stack is standard enough that it's not a blocker when making hires; software engineers either already know the technology, or can pick it up quickly.

Know how different or similar your new business is to other startups. I liked how Tuomas spelled out that Linear is not inventing brand new things. Instead, they are building *better* versions of things. This insight drives their focus and approach to hiring people interested in software craftsmanship, performance and user experience.

Hiring slower could be a sensible strategy for many startups. Tuomas was clearly burnt by Uber's hypergrowth phase, when the number of engineers doubled or tripled every year. As he said, he doesn't want to see that again.

Instead of turning up hiring as the product grows, he and the team have decided to invest early on in their infrastructure, and to accept that they'll build more slowly, but will keep building the right things with a small team.

In recent years, the conventional advice for venture-funded companies was to hire rapidly after raising funds and figure out the rest later. However, Linear's story shows that doing the opposite – hiring slower and growing the team gradually – is a viable strategy.

It's worth noting that while Linear hires slowly, it executes really fast. Just look [at their changelog](#). My theory is that with a smaller, but experienced team which works well together, you can ship just as much – if not more! – than with a much larger team that struggles with communications overhead.

I hope you found the insights into how Linear works and Tuomas's personal journey interesting. Please share how you liked this deep dive. After you tap your feedback, you can leave a text comment, too:

How did you like this newsletter? 🤔

[Amazing](#) • [Great](#) • [Good](#) • [OK](#) • [So-so](#)



90 Likes

7 Comments



Write a comment...



Ishjot Walia Nov 24, 2022 ❤️ Liked by Gergely Orosz

It is a great insight that Big Tech/Big Company experience helps you understand where your technical choices will limit you in the future. It's hard to predict if you haven't seen it before, and can lead to over engineering without the right experience.

♡ LIKE (2) 💬 REPLY ...



Edvins Antonovs Writes Edvins' Newsletter Nov 30, 2022

Great story for sure. I like how Linear focused on quality over quantity despite receiving the investments.

♡ LIKE (1) 💬 REPLY ...

5 more comments...